

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3069 – Computación Paralela y Distribuida

Sección 10

Ing. Kimberly Barrera



Examen Parcial 1 - Consultoría HPC usando OpenMP

Dulce Ambrosio (231143)

Daniel Chet (231177)

Guatemala, 11 de septiembre de 2026

Contexto y Datos

Problema Elegido

Como grupo 6, trabajamos el problema 2, el cual trataba sobre la aproximación del área bajo una curva $f(x)$, compleja y no lineal, en un intervalo $[a, b]$, mediante una suma de Riemann de $n = 10^9$ rectángulos. El problema se ajusta de forma natural al paralelismo de memoria compartida porque la evaluación de cada rectángulo es independiente de las demás, la única dependencia del algoritmo aparece al acumular todas las áreas parciales en un solo total.

Función Integrada y Dominio

La función utilizada combina componentes trigonométricas, exponenciales, de raíz cuadrada y logarítmicas, de manera que cada evaluación individual representa una carga de cómputo no trivial.

$$f(x) = \exp\left(-\frac{x}{10}\right) * \left(\sin^2(x) + \cos^2(2x) + \sqrt{x+1} + \ln(x+1)\right) + \frac{1}{1+x^2}$$

Se integra en el intervalo $[0, 10]$, donde la función es continua, está definida y es estrictamente positiva, por lo que el resultado numérico corresponde a un área geométrica real. El valor de referencia calculado por la versión secuencial y verificado automáticamente por el script de pruebas es 30.80821253638, y todas las corridas paralelas registradas coinciden con este valor hasta el orden de 10^{-11} – 10^{-13} de error relativo.

Representación y Justificación de los Datos de Prueba

No es necesario almacenar los mil millones de valores en memoria, para el rectángulo i se calcula bajo demanda el punto medio del subintervalo correspondiente:

$$h = (b - a) / n$$

$$x_i = a + (i + 0.5) \cdot h$$

$$\text{área} \approx h \cdot \sum f(x_i), \text{ para } i = 0, \dots, n - 1$$

Se eligió la regla del punto medio porque, para una misma cantidad de rectángulos, ofrece generalmente menor error que evaluar en el extremo izquierdo o derecho del subintervalo. Los índices se representan con

enteros de 64 bits (*int64_t*), necesarios porque 10^9 debe seguir siendo válido en todas las plataformas de prueba, y los cálculos se realizan en doble precisión (*double*). La memoria adicional requerida es $O(1)$, no se depende de archivos externos ni de datos generados aleatoriamente, por lo que todas las corridas son perfectamente reproducibles y comparables entre sí. Para el desarrollo incremental del código se usaron corridas rápidas de 2×10^5 rectángulos, la medición final reportada en este informe usa exactamente 10^9 rectángulos.

Propuesta Secuencial

El algoritmo base (carpeta /secuencial) recorre todos los índices de 0 a $n - 1$, evalúa $f(x_i)$ en el punto medio de cada subintervalo, acumula la suma en una variable escalar y, al finalizar el ciclo, multiplica el acumulado por el ancho h para obtener el área total. Su complejidad temporal es $O(n)$ y su memoria adicional es $O(1)$.

El cronómetro (`std::chrono::steady_clock`) mide únicamente el ciclo de integración, sin incluir el análisis de argumentos de línea de comandos ni la impresión del resultado, de modo que el tiempo medido corresponde exclusivamente al trabajo de cómputo.

Estrategia de Paralelización

La versión paralela (carpeta /paralelo) reutiliza exactamente la misma función matemática y la misma regla de integración que la versión secuencial, y añade las siguientes directivas y decisiones de diseño con OpenMP:

- *#pragma omp parallel default(none) shared(...) reduction(+:suma):*

Crea una única vez el equipo de hilos para toda la región paralela, en lugar de crearlo y destruirlo en cada iteración, minimizando el overhead de administración de hilos.

- *default(none):*

Obliga a declarar explícitamente el alcance de cada variable utilizada dentro de la región paralela, lo que ayuda a detectar en tiempo de compilación posibles errores de variables compartidas por accidente.

- *#pragma omp for schedule(static):*

Reparte los bloques de iteraciones del ciclo principal de forma determinista y en partes iguales entre los hilos. Como todas las evaluaciones de $f(x)$ cuestan aproximadamente lo mismo (no hay ramas costosas que dependan del valor de i), un reparto estático ofrece un buen balance de carga con menos overhead que un `schedule(dynamic)`, que sería preferible únicamente si el costo por iteración fuera variable e impredecible.

- *reduction(+:suma):*

Es el mecanismo central para evitar condiciones de carrera. OpenMP crea una copia privada de la variable suma para cada hilo, cada hilo acumula únicamente en su copia local y, al finalizar la región paralela, el runtime combina automáticamente todas las sumas parciales en la variable original. Esto elimina por completo la necesidad de secciones críticas (`critical`) o de operaciones atómicas (`atomic`) dentro del ciclo caliente, cualquiera de esas alternativas serializaría mil millones de actualizaciones sobre una misma variable y destruiría por completo la escalabilidad de la solución.

- *omp_set_dynamic(0) y omp_set_num_threads(hilos):*

Impiden que el runtime de OpenMP ajuste silenciosamente la cantidad de hilos solicitada, lo cual es indispensable para que los experimentos de speedup con una cantidad fija de hilos sean repetibles y comparables.

- *omp_get_wtime()*:

Mide el tiempo de pared (wall-clock) real de la región paralela, incluyendo la creación del equipo de hilos y la reducción final, de forma consistente cómo se mide el tiempo secuencial.

Para p hilos, el trabajo ideal por hilo es aproximadamente n/p , de modo que el tiempo computacional esperado es del orden de $O(n/p)$ más el costo fijo de crear el equipo de hilos y de combinar las p sumas parciales en la reducción final. Cabe resaltar que la suma de números en punto flotante no es asociativa, el orden en que se combinan las sumas parciales puede cambiar según la cantidad de hilos, por lo que el último bit del resultado puede diferir levemente entre configuraciones aunque el algoritmo sea completamente correcto. Por esta razón, la verificación automática (scripts/test.ps1) y el benchmark comparan el área obtenida contra la versión secuencial usando un error relativo (del orden de 10^{-11} a 10^{-13} en las mediciones registradas) en lugar de exigir una igualdad exacta bit a bit.

Metodología Experimental

Cada integrante del equipo realizó las mediciones de forma independiente en su propia computadora, siguiendo el mismo procedimiento para garantizar comparabilidad interna:

- Compilación única de ambas versiones con `-O3 -march=native`, solo la versión paralela agrega `fopenmp` (*scripts/build.ps1*).
- Cada computadora se mantuvo conectada a corriente, con el mismo perfil de energía y sin aplicaciones de carga considerable en ejecución durante las mediciones.
- Se ejecutó una corrida de calentamiento (no registrada) antes de iniciar las mediciones formales.
- Se ejecutaron 10^9 rectángulos con distintas cantidades de hilos, sin superar los hilos lógicos disponibles en cada máquina, con 5 repeticiones por configuración (*scripts/benchmark.ps1*).
- Para cada configuración se reporta la mediana de las 5 repeticiones, por ser menos sensible a interrupciones ocasionales del sistema operativo que el promedio.

El comando utilizado por cada integrante tuvo la forma general:

```
.\scripts\benchmark.ps1 `  
  
-Integrante "nombre_apellido" `  
  
-Rectangulos 1000000000 `
```

-Repeticiones 5`

-Hilos "1,2,4,8,16,..."

Las métricas reportadas se calculan como:

$$speedup(p) = T_{secuencial} / T_{paralelo}(p)$$

$$eficiencia(p) = speedup(p) / p \cdot 100 \%$$

El speedup indica cuántas veces más rápida es la solución paralela con p hilos respecto a la versión secuencial en la misma máquina. La eficiencia indica qué fracción de la capacidad teórica de esos p hilos se está aprovechando, un valor de 100 % representaría un speedup lineal perfecto.

Resultados y Métricas por Integrante

Ambos integrantes verificamos que el área calculada coincidiera con el valor de referencia (30.80821253638) para todas las configuraciones de hilos, con error relativo máximo del orden de 10^{-11} a 10^{-13} , lo que confirma que la reducción de OpenMP elimina correctamente las condiciones de carrera sin sacrificar la exactitud numérica del resultado.

Integrante 1 – Daniel Chet

Rectángulos evaluados	1,000,000,000 (10^9)
Repeticiones por configuración	5 (se reporta la mediana)
Configuraciones de hilos probadas	1, 2, 4, 8, 16, 32
Compilación	g++ -std=c++20 -O3 -march=native (secuencial); + -fopenmp (paralelo)
Detalles de CPU / RAM / SO	Intel i9-13980HX, 24 núcleos / 32 hilos

Hilos	T.secuencial mediana (s)	T.paralelo mediana (s)	Speedup	Eficiencia (%)	Error relativo máximo
1	37.2865	37.2840	1.0001	100.01	0
2	37.2865	19.1680	1.9452	97.26	5.24×10^{-13}
4	37.2865	9.8880	3.7709	94.27	1.98×10^{-12}
8	37.2865	5.4600	6.8290	85.36	1.99×10^{-12}
16	37.2865	3.7840	9.8537	61.59	2.00×10^{-12}
32	37.2865	2.6430	14.1076	44.09	1.99×10^{-12}

Figura 1. Captura de pantalla de la ejecución del benchmark (Daniel Chet)

```

Problems Output Debug Console Ports Terminal
PS C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida> .\scripts\benchmark.ps1 `
>> -Integrante "Daniel Chet" `
>> -Rectangulos 1000000000 `
>> -Repeticiones 5 `
>> -Hilos "1,2,4,8,16,32"
Calentamiento (no se registra)...
Secuencial: repeticion 1 de 5
Secuencial: repeticion 2 de 5
Secuencial: repeticion 3 de 5
Secuencial: repeticion 4 de 5
Secuencial: repeticion 5 de 5
Paralelo (1 hilo(s)): repeticion 1 de 5
Paralelo (1 hilo(s)): repeticion 2 de 5
Paralelo (1 hilo(s)): repeticion 3 de 5
Paralelo (1 hilo(s)): repeticion 4 de 5
Paralelo (1 hilo(s)): repeticion 5 de 5
Paralelo (2 hilo(s)): repeticion 1 de 5
Paralelo (2 hilo(s)): repeticion 2 de 5
Paralelo (2 hilo(s)): repeticion 3 de 5
Paralelo (2 hilo(s)): repeticion 4 de 5
Paralelo (2 hilo(s)): repeticion 5 de 5
Paralelo (4 hilo(s)): repeticion 1 de 5
Paralelo (4 hilo(s)): repeticion 2 de 5
Paralelo (4 hilo(s)): repeticion 3 de 5
Paralelo (4 hilo(s)): repeticion 4 de 5
Paralelo (4 hilo(s)): repeticion 5 de 5
Paralelo (8 hilo(s)): repeticion 1 de 5
Paralelo (8 hilo(s)): repeticion 2 de 5
Paralelo (8 hilo(s)): repeticion 3 de 5
Paralelo (8 hilo(s)): repeticion 4 de 5
Paralelo (8 hilo(s)): repeticion 5 de 5
Paralelo (16 hilo(s)): repeticion 1 de 5
Paralelo (16 hilo(s)): repeticion 2 de 5
Paralelo (16 hilo(s)): repeticion 3 de 5
Paralelo (16 hilo(s)): repeticion 4 de 5
Paralelo (16 hilo(s)): repeticion 5 de 5
Paralelo (32 hilo(s)): repeticion 1 de 5
Paralelo (32 hilo(s)): repeticion 2 de 5
Paralelo (32 hilo(s)): repeticion 3 de 5
Paralelo (32 hilo(s)): repeticion 4 de 5

```

Figura 2. Captura de pantalla del resumen final de resultados (medianas de 5 repeticiones) (Daniel Chet)

```

Problems Output Debug Console Ports Terminal
PS C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida> .\scripts\benchmark.ps1 `
>> -Integrante "Daniel Chet" `
>> -Rectangulos 1000000000 `
>> -Repeticiones 5 `
>> -Hilos "1,2,4,8,16,32"
Paralelo (16 hilo(s)): repeticion 5 de 5
Paralelo (32 hilo(s)): repeticion 1 de 5
Paralelo (32 hilo(s)): repeticion 2 de 5
Paralelo (32 hilo(s)): repeticion 3 de 5
Paralelo (32 hilo(s)): repeticion 4 de 5
Paralelo (32 hilo(s)): repeticion 5 de 5

Resumen de Daniel_Chet (medianas de 5 repeticiones):

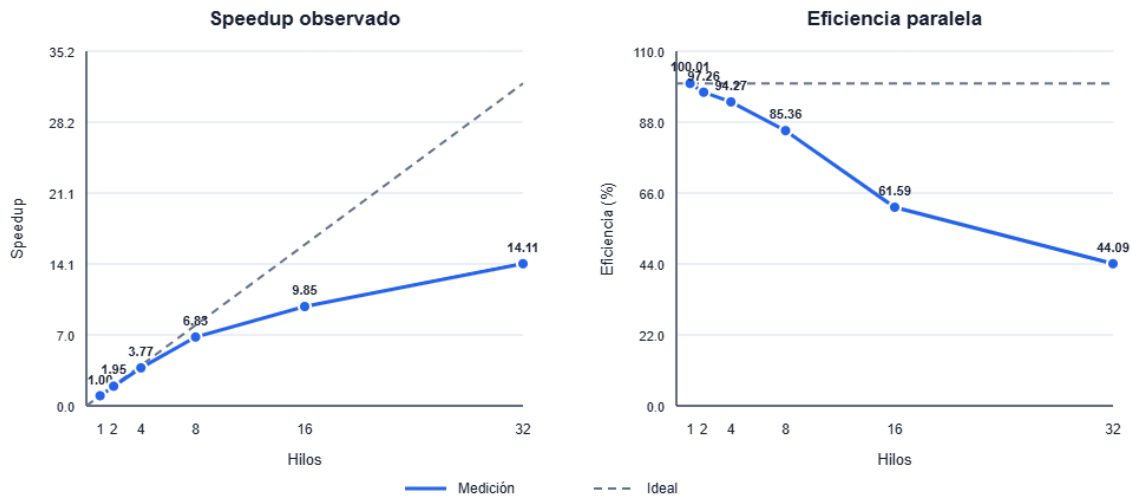
hilos tiempo_secuencial_mediana_s tiempo_paralelo_mediana_s speedup eficiencia porcentaje_error_relativo_maximo
-----
1 37.29 37.28 1.00 100.01 0.00
2 37.29 19.17 1.95 97.26 0.00
4 37.29 9.89 3.77 94.27 0.00
8 37.29 5.46 6.83 85.36 0.00
16 37.29 3.78 9.85 61.59 0.00
32 37.29 2.64 14.11 44.09 0.00

Corridas: C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida\docs\resultados\Daniel_Chet_corridas.csv
Resumen: C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida\docs\resultados\Daniel_Chet_resumen.csv
PS C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida> py .\scripts\graficar_resultados.py `
>> -docs\resultados\Daniel_Chet_resumen.csv `
>> --output .\docs\graficas\Daniel_Chet.svg
Gráfica creada en docs\graficas\Daniel_Chet.svg
PS C:\UVG\Semestre8\CC - Programacion Paralela y Distribuida\Parcial\Parcial1\CC3069-Computacion-Paralela-y-Distribuida>

```

Figura 3. Gráfica de speedup observado y eficiencia paralela vs. cantidad de hilos (Daniel Chet)

Resultados de OpenMP — Daniel_Chet



El mejor tiempo de Daniel Chet se obtuvo con 32 hilos, logrando un speedup de $14.11\times$ y una eficiencia de 44.09 %. Con 1 hilo, el tiempo paralelo (37.28 s) es prácticamente idéntico al secuencial (37.29 s), lo que muestra que el overhead de crear el equipo de OpenMP es despreciable frente al tamaño del problema.

Integrante 2 – Dulce Ambrosio

Rectángulos evaluados	1,000,000,000 (10^9)
Repeticiones por configuración	5 (se reporta la mediana)
Configuraciones de hilos probadas	1, 2, 4, 8, 16, 20
Compilación	g++ -std=c++20 -O3 -march=native (secuencial); + -fopenmp (paralelo)
Detalles de CPU / RAM / SO	Procesador 12 Gen Intel Core i7, RAM 8,

	Núcleos 14 y 20 hilos
--	-----------------------

Hilos	T.secuencial mediana (s)	T.paralelo mediana (s)	Speedup	Eficiencia (%)	Error relativo máximo
1	24.5095	24.4800	1.0012	100.12	0
2	24.5095	12.6030	1.9447	97.24	5.24×10^{-13}
4	24.5095	6.2690	3.9096	97.74	1.98×10^{-12}
8	24.5095	3.8260	6.4060	80.08	1.99×10^{-12}
16	24.5095	2.8490	8.6028	53.77	2.00×10^{-12}
20	24.5095	2.7170	9.0208	45.10	1.98×10^{-11}

Figura 1. *Captura de pantalla de la ejecución del benchmark (Dulce Ambrosio)*

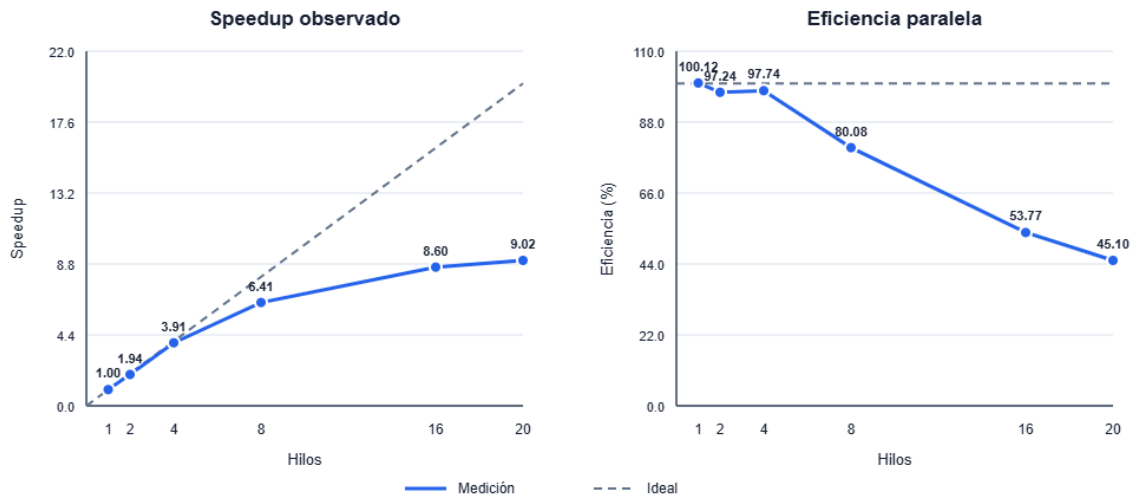
```
Integrante: Dulce Ambrosio
Hilos logicos detectados: 20
Configuraciones a medir: 1, 2, 4, 8, 16, 20
Rectangulos: 1000000000 | Repeticiones: 5
No uses programas pesados mientras se ejecuta el benchmark.
Calentamiento (no se registra)...
Secuencial: repeticion 1 de 5
Secuencial: repeticion 2 de 5
Secuencial: repeticion 3 de 5
Secuencial: repeticion 4 de 5
Secuencial: repeticion 5 de 5
Paralelo (1 hilo(s)): repeticion 1 de 5
Paralelo (1 hilo(s)): repeticion 2 de 5
Paralelo (1 hilo(s)): repeticion 3 de 5
Paralelo (1 hilo(s)): repeticion 4 de 5
Paralelo (1 hilo(s)): repeticion 5 de 5
Paralelo (2 hilo(s)): repeticion 1 de 5
Paralelo (2 hilo(s)): repeticion 2 de 5
Paralelo (2 hilo(s)): repeticion 3 de 5
Paralelo (2 hilo(s)): repeticion 4 de 5
Paralelo (2 hilo(s)): repeticion 5 de 5
Paralelo (4 hilo(s)): repeticion 1 de 5
Paralelo (4 hilo(s)): repeticion 2 de 5
Paralelo (4 hilo(s)): repeticion 3 de 5
Paralelo (4 hilo(s)): repeticion 4 de 5
Paralelo (4 hilo(s)): repeticion 5 de 5
Paralelo (8 hilo(s)): repeticion 1 de 5
Paralelo (8 hilo(s)): repeticion 2 de 5
Paralelo (8 hilo(s)): repeticion 3 de 5
Paralelo (8 hilo(s)): repeticion 4 de 5
Paralelo (8 hilo(s)): repeticion 5 de 5
Paralelo (16 hilo(s)): repeticion 1 de 5
Paralelo (16 hilo(s)): repeticion 2 de 5
Paralelo (16 hilo(s)): repeticion 3 de 5
Paralelo (16 hilo(s)): repeticion 4 de 5
Paralelo (16 hilo(s)): repeticion 5 de 5
Paralelo (20 hilo(s)): repeticion 1 de 5
Paralelo (20 hilo(s)): repeticion 2 de 5
Paralelo (20 hilo(s)): repeticion 3 de 5
Paralelo (20 hilo(s)): repeticion 4 de 5
Paralelo (20 hilo(s)): repeticion 5 de 5
```

Figura 2. Captura de pantalla del resumen final de resultados (medianas de 5 repeticiones) (Dulce Ambrosio)

hilos	tiempo_secuencial_mediana_s	tiempo_paralelo_mediana_s	speedup	eficiencia_porcentaje	error_relativo_maximo
1	24.5094847	24.4799997806549	1.00120444933044	100.120444933044	0
2	24.5094847	12.6029999256134	1.94473417794669	97.2367088973346	5.23654944159792E-13
4	24.5094847	6.26899981498718	3.90963238528188	97.740809632047	1.98253155692472E-12
8	24.5094847	3.82599997520447	6.40603367978072	80.0754209972589	1.99175692480245E-12
16	24.5094847	2.8490002155304	8.6028370817224	53.767731760765	1.9952164377566E-12
20	24.5094847	2.71700000762939	9.02078933793774	45.1039466896887	1.98091711754612E-12

Figura 3. Gráfica de speedup observado y eficiencia paralela vs. cantidad de hilos (Dulce Ambrosio)

Resultados de OpenMP — Dulce_Ambrosio



El mejor tiempo de Dulce Ambrosio se obtuvo con el máximo de 20 hilos disponibles en su máquina, logrando un speedup de 9.02× y una eficiencia de 45.10 %. Al igual que en el caso anterior, el tiempo con 1 hilo (24.48 s) es esencialmente igual al secuencial (24.51 s), confirmando un overhead de arranque de OpenMP insignificante para este tamaño de problema.

Análisis de Resultados

Comportamiento del speedup y la eficiencia

En ambas máquinas se observa el mismo patrón cualitativo, el speedup crece de forma casi lineal en las primeras duplicaciones de hilos (1, 2, 4), con eficiencias superiores al 94 % en ambos casos hasta 4 hilos, y comienza a alejarse de la curva ideal a partir de 8 hilos, degradándose de forma más marcada al acercarse a

la cantidad máxima de hilos lógicos de cada equipo (85.36 %, 61.59 %, 44.09 % para Daniel Chet entre 8, 16 y 32 hilos; 80.08 %, 53.77 %, 45.10 % para Dulce Ambrosio entre 8, 16 y 20 hilos). Esta caída de eficiencia no significa que la implementación sea incorrecta, es el comportamiento esperado por varias

razones combinadas, como el costo fijo de crear y coordinar el equipo de hilos y de combinar las sumas parciales en la reducción se vuelve relativamente más significativo cuantos más hilos participan; existe competencia por recursos compartidos del procesador como el ancho de banda de memoria caché y unidades de ejecución y si la cantidad de hilos solicitados se acerca o supera la cantidad de núcleos físicos, el hardware multithreading (SMT/Hyper-Threading) comparte unidades de ejecución entre hilos lógicos del mismo núcleo físico y no ofrece el mismo beneficio marginal que un núcleo físico adicional. Esto es coherente con la caída más pronunciada de eficiencia observada en ambas máquinas justo en las configuraciones de mayor cantidad de hilos, que probablemente exceden la cantidad de núcleos físicos disponibles.

Overhead de OpenMP con un solo hilo

Comparar $T_{\text{paralelo}}(1)$ contra $T_{\text{secuencial}}$ permite cuantificar el overhead puro de OpenMP, sin beneficio de paralelismo real. En el caso de Daniel Chet, $T_{\text{secuencial}} = 37.2865$ s y $T_{\text{paralelo}}(1) = 37.2840$ s (una diferencia de apenas 0.0025 s, speedup ≈ 1.0001). En el caso de Dulce Ambrosio, $T_{\text{secuencial}} = 24.5095$ s y $T_{\text{paralelo}}(1) = 24.4800$ s (diferencia de 0.0295 s, speedup ≈ 1.0012).

En ambos casos el overhead de crear la región paralela y hacer la reducción con un solo hilo es prácticamente nulo frente al tiempo total de cómputo, lo que confirma que, para cargas de trabajo grandes como 10^9 rectángulos, ese costo fijo queda ampliamente amortizado por el trabajo útil. Para cargas mucho más pequeñas ese mismo overhead relativo podría hacer que la versión secuencial fuera preferible a la paralela.

Comparación entre las dos máquinas

El equipo de Dulce Ambrosio resuelve la versión secuencial en menos tiempo (24.51 s) que el de Daniel Chet (37.29 s), lo que sugiere un procesador con mayor desempeño por núcleo (frecuencia y/o microarquitectura) en el caso de Dulce Ambrosio. La máquina de Daniel Chet fue capaz de escalar exitosamente hasta 32 hilos y alcanzó el mayor speedup absoluto del equipo ($14.11\times$), mientras que la máquina de Dulce Ambrosio, con 20 hilos lógicos detectados como máximo, alcanzó un speedup máximo de $9.02\times$. Esto es consistente con que Daniel Chet dispone de una mayor cantidad de hilos lógicos para repartir el mismo trabajo, lo que naturalmente permite un mayor speedup potencial, aunque a costa de una eficiencia marginal decreciente conforme se acerca al límite de hilos de su hardware. En ambas máquinas la eficiencia final ronda un rango similar (44–45 %) en su configuración de mayor cantidad de hilos, lo que indica un patrón de saturación comparable una vez que se agota el paralelismo físico disponible en cada equipo.

Conclusiones

- La independencia entre rectángulos en la suma de Riemann permite repartir el ciclo principal entre hilos sin ninguna comunicación durante el cálculo, la única dependencia global (la suma total) se resuelve de forma segura y eficiente con `reduction(+:suma)`, sin necesidad de secciones críticas ni operaciones atómicas.
- `schedule(static)` fue la decisión correcta para esta carga de trabajo, ya que todas las iteraciones tienen un costo de cómputo prácticamente uniforme, por lo que un reparto estático en bloques iguales evita el overhead adicional de un scheduling dinámico sin sacrificar balance de carga.
- La solución paralela logró mejoras sustanciales y correctas, un speedup máximo de $14.11\times$ (44.09 % de eficiencia) con 32 hilos para Daniel Chet, y de $9.02\times$ (45.10 % de eficiencia) con 20 hilos para Dulce Ambrosio, siempre manteniendo un error relativo frente a la versión secuencial del orden de 10^{-11} a 10^{-13} , es decir, sin ningún costo de exactitud numérica.
- La eficiencia decrece de forma consistente al aumentar la cantidad de hilos en ambas máquinas, especialmente al acercarse a la cantidad total de hilos lógicos disponibles, lo cual es el comportamiento esperado por el costo fijo de coordinación, la competencia por recursos compartidos del procesador y las limitaciones del hardware multithreading (SMT), y no un defecto de la implementación.
- Las diferencias de hardware entre las dos computadoras del equipo (una con mayor desempeño secuencial por núcleo, la otra con mayor cantidad de hilos

lógicos disponibles) explican por qué cada una alcanza un speedup máximo distinto, reforzando la importancia de nunca comparar tiempos absolutos entre máquinas distintas y sí comparar cada resultado paralelo contra su propia línea base secuencial.

Repositorio

El código realizado se encuentra en el siguiente [enlace](#) (click en la imagen).

